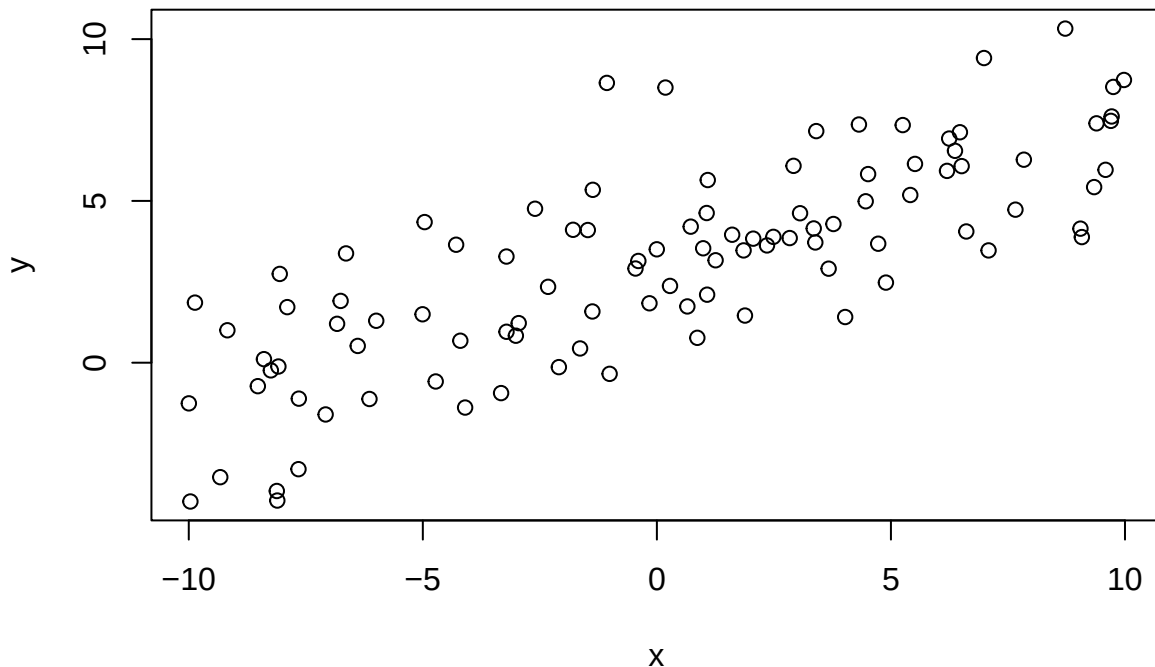


Learning Maximum Likelihood & Bayesian Inference

Let's try to understand how we can estimate statistical parameters using Maximum Likelihood and Bayesian Inference.

Now let's turn to a linear regression. Let's first generate data that follows a standard linear regression model, which should be familiar to you as $y = m * x + b + \epsilon$. x can take on any set of values, while m and b are regression parameters. ϵ is a random error term that we will model as a normal distribution with mean $\mu = 0$ and σ^2 , which will be estimated from the data. Let's begin by simulating the data.

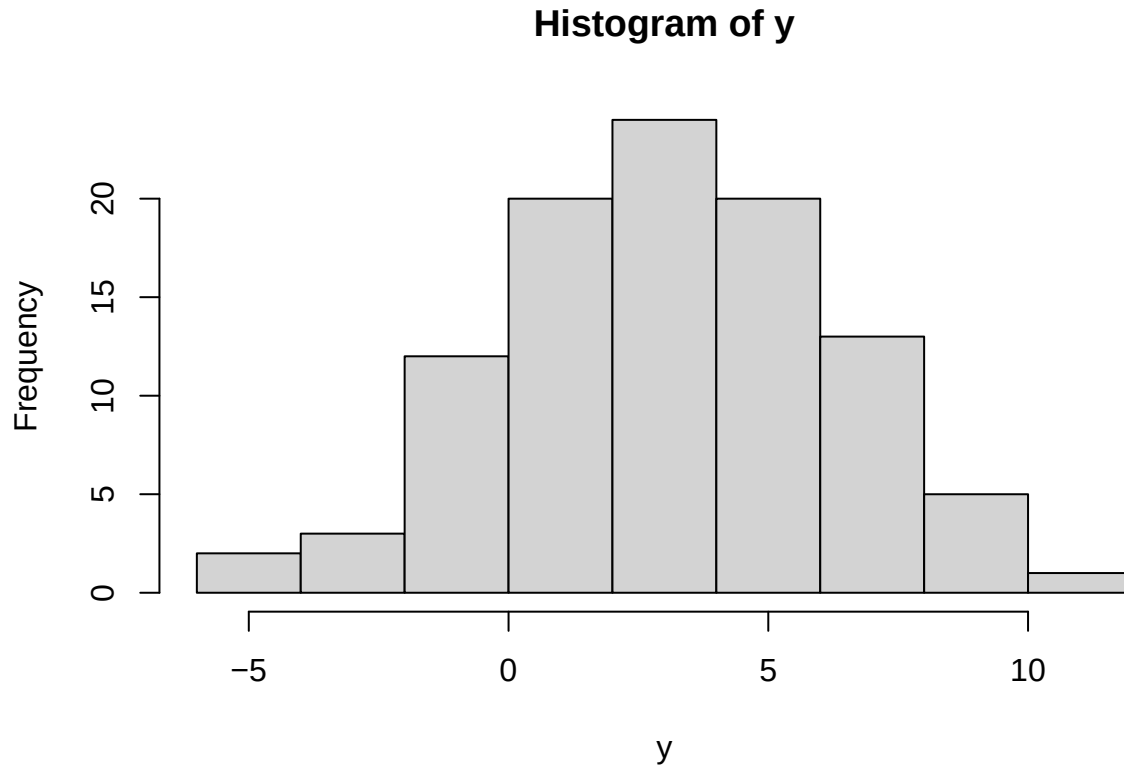
```
n <- 100 # The number of data points to simulate
m <- 0.5 #Pick a value for m, this is the slope of the regression
b <- 3 #Pick a value for b, this is the intercept
sig2 <- 3 #Pick a [positive] value for sigma^2, this is the variance of the normal error
x <- runif(n, -10, 10)
y <- m*x + b + rnorm(n, 0, sqrt(sig2))
plot(x,y)
```



Now, we could estimate this regression using R's built in tools, but we want to learn how to do it ourselves using Maximum Likelihood. What is a likelihood? It is simply the probability of observing the data given certain parameters $P(\text{data}|\text{parameters})$. Our goal for maximizing the likelihood is to simply search for the parameters that make $P(\text{data}|\text{parameters})$ as big as it can possibly be. How do we calculate that probability?

Let's start by assuming that $m = 0$ and $b = 0$, i.e. that the data are simple draws from a normal distribution. Then we could use the R function `dnorm`, which provides the *probability density* of a particular data point.

```
##?dnorm ##Get help
## Calculate the probability of observing y for sigma^2 = 1
hist(y)
```



```
L <- dnorm(y, 0, 1)
L
```

```
## [1] 1.059011e-17 3.965398e-01 6.447887e-02 1.939128e-01 8.989596e-06
## [6] 4.579060e-05 2.396593e-04 1.525914e-01 1.721413e-01 6.760795e-17
## [11] 5.779951e-05 4.090522e-05 1.578115e-06 1.329552e-03 9.293962e-06
## [16] 1.380771e-01 1.139605e-01 4.850728e-06 2.240882e-20 2.414639e-01
## [21] 3.853918e-12 1.128702e-09 2.317120e-17 2.556509e-02 8.799565e-02
## [26] 1.070734e-04 3.370674e-01 1.943326e-10 2.392777e-02 7.683728e-04
## [31] 2.968558e-01 7.120544e-02 2.152911e-01 5.849182e-07 5.182639e-04
## [36] 2.964835e-13 3.878557e-01 2.565934e-01 1.608318e-07 2.070725e-04
## [41] 1.514295e-11 2.126432e-01 4.802968e-08 3.759664e-01 6.834468e-13
## [46] 4.376597e-02 3.488065e-01 3.002310e-12 7.630112e-04 8.940086e-05
## [51] 3.066645e-01 3.850241e-09 1.674899e-08 5.243429e-13 7.970171e-17
## [56] 7.474554e-05 2.827504e-24 7.389235e-02 1.472537e-01 4.617648e-04
## [61] 3.961772e-01 5.772706e-03 3.669032e-09 1.537390e-04 1.302392e-01
## [66] 2.817816e-01 3.620902e-01 4.033777e-05 5.610719e-04 3.999437e-04
## [71] 8.661952e-05 7.685894e-09 1.109196e-01 1.862170e-02 2.565003e-04
## [76] 1.059478e-13 9.267503e-03 3.161436e-05 3.952099e-01 1.777773e-03
## [81] 7.286917e-05 9.674585e-04 5.547101e-06 2.846895e-03 7.767170e-13
```

```
## [86] 9.128361e-02 2.547203e-09 3.168808e-01 9.345231e-09 2.527309e-01
## [91] 1.811672e-01 2.662309e-03 1.833627e-03 9.690545e-04 1.600216e-04
## [96] 1.887988e-01 2.124374e-04 2.523073e-07 8.591369e-04 5.848982e-03
```

Notice that many numbers are VERY SMALL. Computationally, it works better if we take the log of the probability density

```
logLs <- log(L)
```

Now we take the sum of this number to get the log-Likelihood of the data given $m=0$, $b=0$ and $\sigma^2 = 1$.

```
logL <- sum(logLs)
logL
```

```
## [1] -1077.663
```

However, we know these are not the parameters we used to generate these data. What's the log likelihood under those parameters? Well we simply need to define for each data point, it's expectation and variance. In other words, we need to calculate the residuals from the model, then plug them into `dnorm`.

```
expected_values <- m*x + b
residuals <- y - expected_values
logL <- sum(dnorm(residuals, 0, sqrt(sig2), log=TRUE))
logL
```

```
## [1] -214.036
```

Notice that while the log-likelihood is still small, it is substantially larger than it was previously. But is it the maximum? Let's make a function of the likelihood to streamline calculating the likelihood for a set of parameters.

```
logL_fx <- function(parameters, x, y){
  m <- parameters[1]
  b <- parameters[2]
  sig2 <- parameters[3]
  expected_values <- m*x + b
  residuals <- y - expected_values
  logL <- sum(dnorm(residuals, 0, sqrt(sig2), log=TRUE))
  return(logL)
}
```

Verify that it returns the same value as before:

```
logL_fx(c(m, b, sig2), x, y)
```

```
## [1] -214.036
```

Now we can use computational tools to *optimize* the best likelihood. These are “hill-climbing” algorithms.

```
starting_values <- c(0, 0, 1) #Our initial guesses for the parameters
optim(par=starting_values, fn=logL_fx, control=list(fnscale=-1), x=x, y=y)
```

```
## $par
## [1] 0.4257793 3.0711437 3.8477963
##
## $value
## [1] -209.2617
##
## $counts
## function gradient
```

```
##      140      NA
##
## $convergence
## [1] 0
##
## $message
## NULL
```

Compare to R's built-in regression estimation:

```
lm1 <- lm(y~x)
summary(lm1)
```

```
##
## Call:
## lm(formula = y ~ x)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -3.8772 -1.4635  0.0261  1.2219  6.0314
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  3.07064    0.19822   15.49  <2e-16 ***
## x            0.42584    0.03444   12.37  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.981 on 98 degrees of freedom
## Multiple R-squared:  0.6094, Adjusted R-squared:  0.6054
## F-statistic: 152.9 on 1 and 98 DF,  p-value: < 2.2e-16
sqrt(sig2)
```

```
## [1] 1.732051
```

Likely, the biggest difference will be in the residual standard error. This is because Maximum Likelihood estimators are biased estimators, whereas the algorithm used in R is an unbiased estimator of the variance. Nevertheless, this bias will get smaller and smaller as more data are added.

Congratulations! That's all there is to ML estimation. In phylogenetics, we do the same thing, except instead of m , b and σ^2 , we are estimating τ (the topology of the tree), ν (a vector of branch lengths), and Q (a matrix of transition probabilities) and our data are gene sequences or phenotypes.

##What about Bayesian analyses?##

Suppose you're an alien visiting earth for the first time. Not knowing any borders or habits of the people who live there, you randomly chose a particular human to suck up and analyze. You find that this particular human is wearing what they call a cowboy hat. You also learn of a people known as "Texans" that often wear cowboy hats. You hypothesize that given the observed data (human X is wearing a cowboy hat) that human X is a Texan. So long as $P(\text{Cowboyhat}|\text{Texan}) > P(\text{Cowboyhat}|\text{anyotherstate})$, then the Maximum Likelihood conclusion that you should make is that the human you obtained is in fact, a Texan (even if Wyomingites and Montanans are only slightly less likely to wear cowboy hats). Suppose that the rate of cowboy hat wearing in Texas is 10%.

But this is odd. What we really want to know is what is the probability that our human is in fact, a Texan. Instead, ML has given us "What are the most likely people to wear cowboy hats?". Mathematically, the rules of conditional probability relate these two quantities for us:

$$P(\text{Cowboyhat}|\text{Texan})P(\text{Texan}) = P(\text{Texan}|\text{Cowboyhat})P(\text{Cowboyhat})$$

So we figured out how to maximize $P(\text{Cowboyhat}|\text{Texan})$, but we really want $P(\text{Texan}|\text{Cowboyhat})$. We can do some algebra to figure out what we need. It turns out it is: $P(\text{Texan}|\text{Cowboyhat}) = \frac{P(\text{Texan})P(\text{Cowboyhat}|\text{Texan})}{P(\text{Cowboyhat})}$

This is Bayes Formula.

Let's go through each of these terms. $P(\text{Texan})$ is the probability that a random human from the world is Texan. We know this value, it is approximately 28.3 million/7.6 billion or ~0.003724. What's the Probability of someone wearing a cowboy hat? That's a harder number to estimate. But we could go around estimating it for each state/country, e.g. $P(\text{cowboyhat}|\text{Virginian})$ and $P(\text{cowboyhat}|\text{Norwegian})$ wouldn't be too big, while $P(\text{cowboyhat}|\text{Montanan})$, $P(\text{cowboyhat}|\text{Argentinian})$ and $P(\text{cowboyhat}|\text{Wyomingite})$ might be a bit bigger. Of course for each one, we would have to also weight them by their population.

In the end, we'd get this: $P(\text{Texan}|\text{Cowboyhat}) = \frac{0.003724 \cdot P(\text{Texan})}{P(\text{cowboyhat}|\text{Montanan}) \cdot P(\text{Montanan}) + P(\text{cowboyhat}|\text{Virginian}) \cdot P(\text{Virginian}) + P(\text{cowboyhat}|\text{Norwegian}) \cdot P(\text{Norwegian}) + P(\text{cowboyhat}|\text{Wyomingite}) \cdot P(\text{Wyomingite})}$

Notice that the denominator is a daunting quantity to calculate. Let's instead limit our world to a few regions. The alien knows that the human was sucked up from either Texas, Montana, California, or Virginia with populations of 28.3 million, 1.1 million, 39.5 million and 8.5 million respectively. Furthermore, let's assume they have cowboy hat wearing at rates of 10%, 9%, 1% and 0.01%. Under ML, it's clear that Texans are most likely to wear cowboy hats. However, let's plug it into Bayes Formula.

```
pop <- c("Texas"=28.3, "Montana"=1.1, "California"=39.5, "Virginia"=8.5)
totalpop <- sum(pop)
pState <- pop/totalpop
cat("\n Prior probabilities of being an inhabitant of each state\n")
```

```
##
## Prior probabilities of being an inhabitant of each state
pState

##      Texas      Montana California  Virginia
## 0.36563307 0.01421189 0.51033592 0.10981912
```

Now calculate the Likelihoods for each state.

```
liks <- c("Texas"=10, "Montana"=9, "California"=1, "Virginia"=0.01)/100
```

Followed by the denominator of Bayes Formula, or the marginal probability of wearing a cowboy hat.

```
denom <- pState * liks
```

Put it all together in Bayes Formula:

```
post <- (liks['Texas']*pState['Texas'])/sum(denom)
post
```

```
##      Texas
## 0.8511662
```

This is the *posterior probability* that someone is a Texan if you observe they are wearing a cowboy hat. We can see that our certainty of them being Texan went up from our prior probability:

```
#Prior probability
pState["Texas"]
```

```
##      Texas
## 0.3656331
```

```
## Increase in posterior over prior
post/pState["Texas"]
```

```
##      Texas
## 2.327925
```

Furthermore, our human being is still the most probably from Texas. However, suppose that ALL Montanans wear Cowboy hats. What is our posterior probability of our person being a Cowboy then? Under Maximum Likelihood, we would definitely conclude that our human was a Montanan. However, under Bayesian Inference we would STILL conclude that they were most likely a Texan

```
liks["Montana"] <- 1
denom <- sum(pState*liks)
post <- pState['Texas']*liks['Texas']/denom
post
```

```
##      Texas
## 0.6542067
```

Why? Well because we knew beforehand that they were very unlikely to be a Montanan! Compare the prior and posterior of being from each state:

```
posts <- pState*liks/denom
results <- data.frame("prior"=round(pState,2), "likelihood"=round(liks,2), "posterior"=round(posts,2))
results
```

```
##           prior likelihood posterior
## Texas      0.37      0.10      0.65
## Montana    0.01      1.00      0.25
## California 0.51      0.01      0.09
## Virginia   0.11      0.00      0.00
```

The posterior probabilities went up a lot for us having a Montanan! 25x greater. However, even if every Montanan wears a cowboy hat, we know there are so few of them that it is more likely they come from another, more populous state.

This is the heart of Bayesian Analyses, with the main difference being that we include a prior. Sometimes those priors are well-justified. For example, our priors here are based on knowledge of human population sizes. However, if you didn't have that available, the results would end up looking almost identical to Maximum Likelihood analysis:

```
pState_uninformative <- rep(0.25,4)
denom2 <- sum(pState_uninformative*liks)
posts2 <- pState_uninformative*liks/denom2
results_uninformative <- data.frame("prior"=pState_uninformative, "likelihood"=round(liks,2), "posterior"=round(posts2,2))
results_uninformative
```

```
##           prior likelihood posterior
## Texas      0.25      0.10      0.09
## Montana    0.25      1.00      0.90
## California 0.25      0.01      0.01
## Virginia   0.25      0.00      0.00
```

So Bayesian analysis does a better job of answering what questions we probably want answers to, but can be harder to apply for two reasons: 1) The prior may be hard to justify and 2) the denominator of Bayes' formula is hard to figure out! For (1), we can either try to justify our prior using data, or we can see whether our conclusions are *prior sensitive* by trying a range of plausible prior values. If our results are unaffected by our choice of prior, then we don't have to worry! For (2), we often make use a computational trick called Markov Chain Monte Carlo.

A Simple MCMC

```
#We're going to travel randomly between all 4 states. Let's start in Texas.
gens <- 10000 # Pick how many times you want to try moving between states
states <- names(pState)
state <- rep(NA, gens)
state[1] <- "Texas"
lik <- rep(NA, gens)
prior <- rep(NA, gens)
lik[1] <- liks[state[1]]
prior[1] <- pState[state[1]]

for(i in 2:gens){
  proposed_state <- sample(states, 1) #Propose a new state at random
  num_post_state_old <- prior[i-1]*lik[i-1] # get the posterior numerator for the old state
  num_post_state_new <- pState[proposed_state]*liks[proposed_state] # get the posterior numerator for the new state
  if(num_post_state_new > num_post_state_old){ #If the new state has a higher posterior probability, accept it
    state[i] <- proposed_state
    lik[i] <- liks[proposed_state]
    prior[i] <- pState[proposed_state]
  } else{ #If it's not higher, accept it anyway with probability proportional to the ratio of the probabilities
    u <- runif(1)
    a <- num_post_state_new/num_post_state_old
    if(u < a){
      state[i] <- proposed_state
      lik[i] <- liks[proposed_state]
      prior[i] <- pState[proposed_state]
    } else {
      state[i] <- state[i-1]
      lik[i] <- lik[i-1]
      prior[i] <- prior[i-1]
    }
  }
}
```

```
MCMCEstimates <- table(factor(state,states))/gens
```

```
data.frame(results, "mcmcEst"=round(as.vector(MCMCEstimates),2))
```

```
##           prior likelihood posterior mcmcEst
## Texas      0.37      0.10      0.65      0.65
## Montana    0.01      1.00      0.25      0.26
## California 0.51      0.01      0.09      0.09
## Virginia   0.11      0.00      0.00      0.00
```

In phylogenetics, we're just doing the same exact thing as we've done above. Instead of $P(\text{state} \mid \text{cowboy hat})$ we have $P(\text{tree} \mid \text{sequence/phenotypic data})$. Tree space is a lot bigger than 4 states, it's enormous, and the likelihoods are a bit trickier to calculate, but doable. What priors do we use in a Bayesian analysis? That's a tricky question and the subject of much debate. Ideally, we'd use informative priors. In practice, people usually use software defaults. This is bad. Don't do this. Learn about what priors mean. And when you review papers, make people 1) specify what priors they used for every parameter and 2) make people justify their use of those priors.

##Breast Cancer Testing## Why are you not recommended to get a mammogram until you're under 40?

Research on breast cancer has determined the following statistics (I don't think these are actually accurate, but they convey the idea):

1% of women have breast cancer 99% of mammograms detect breast cancer when breast cancer is present 2% of mammograms detect breast cancer when breast cancer IS ABSENT

Imagine you received a positive mammogram test stating that breast cancer is detected. How worried should you be?

Make a statement using conditional probability that represents the probability you want to determine.

Then use Bayes' Formula to determine that conditional probability. Compare to the answer from Maximum Likelihood.